Checkpoint D — Architecture and Changeability

QuantRule

2026-11-11

Contents

1 Checkpoint D — Architecture and Changeability **1**
- 1.1 Claim . . . 1
- 1.2 Current architecture . . . 1
- 1.3 Change scenarios . . . 1
- 1.4 Evidence . . . 2
- 1.5 Limit . . . 2
- 1.6 Decision . . . 2

1 Checkpoint D — Architecture and Changeability

1.1 Claim

The local-first architecture keeps the deterministic engine isolated from the desktop UI and the distribution channel, so a model, strategy, or UI change stays local and testable.

1.2 Current architecture

- One local quantum: Tauri UI + application service + pure engines.
- Pipes-and-filters inside a modular monolith.
- CLI and UI are adapters over the same application service.
- The download page is a static distribution channel.
- The research memory is a SQLite store behind the application service.

1.3 Change scenarios

Change	Boundary	Evidence
New model reading	`crates/quantrule-model`	golden, causality, determinism tests
New strategy rule	`crates/quantrule-backtest`	strategy boundary tests
UI screen	`apps/desktop/src`	unit and browser E2E
New command	`crates/quantrule-shell`	command tests and help
New installer	`site/`	clean install and checksums
Team cloud (future)	new control plane	not implemented; new ADR

1.4 Evidence

- Crate boundaries and structural checks.
- Model/strategy boundary tests.
- The Tauri UI calls `quantrule-app` through typed commands; it does not import an engine directly.
- The Phase 3 verification report records the clean install, offline launch, and memory round-trips.

1.5 Limit

The UI does not run backtests or analytics; those remain CLI operations. The team cloud is a future quantum and is not covered by this evidence.

1.6 Decision

Keep the pipeline/modular-monolith style; do not add a server or microservices for the solo increment.